

Gestión Segura de Credenciales y Protección de Secretos en OmniSens Industrial Suite

Introducción

En los sistemas IoT industriales, las credenciales representan uno de los activos más sensibles de la infraestructura tecnológica. Contraseñas de bases de datos, claves de acceso MQTT, tokens de APIs y certificados digitales permiten la autenticación y comunicación entre los distintos componentes del sistema.

La exposición accidental de estas credenciales dentro del código fuente constituye una de las vulnerabilidades más frecuentes en proyectos de software. Si una credencial es publicada en un repositorio o compartida de forma insegura, puede ser utilizada por terceros para acceder a servicios críticos, comprometer la integridad de los datos o interrumpir la operación del sistema.

Por este motivo, OmniSens Industrial Suite adopta una estrategia de gestión segura de secretos orientada a eliminar credenciales hardcodeadas, utilizar variables de entorno y aplicar mecanismos automáticos de detección de información sensible.

Objetivo

Prevenir la exposición de información sensible dentro del código fuente mediante la identificación, protección y gestión adecuada de credenciales utilizadas por los distintos componentes del sistema.

Identificación de Credenciales Críticas

Durante el análisis de la arquitectura de OmniSens se identificaron diversos elementos que requieren protección especial.

Base de Datos MariaDB

Información sensible:

- Usuario administrador.

- Contraseñas de acceso.
- Cadenas de conexión.
- Configuración de servicios.

Broker MQTT

Información sensible:

- Usuarios MQTT.
- Contraseñas MQTT.
- Certificados de autenticación.
- Claves privadas.

APIs y Servicios

Información sensible:

- Tokens de acceso.
- Claves API.
- Credenciales de autenticación.
- Tokens de sesión.

Infraestructura Docker

Información sensible:

- Variables de configuración.
- Secretos de despliegue.
- Parámetros de conexión.

Eliminación de Credenciales Hardcodeadas

Las credenciales hardcodeadas son aquellas que se encuentran escritas directamente dentro del código fuente.

Ejemplo inseguro:

```
mqtt_password = "admin123"
```

Este enfoque presenta riesgos significativos, especialmente cuando el código es almacenado en repositorios compartidos.

Para mitigar este problema se establece como política:

- No almacenar contraseñas dentro del código.
- No publicar tokens en repositorios.
- No incluir claves en archivos compartidos.
- Mantener configuraciones sensibles fuera del código fuente.

Uso de Variables de Entorno

Las variables de entorno permiten separar la configuración sensible del código de la aplicación.

Ejemplo conceptual:

```
MQTT_USER=usuario
MQTT_PASSWORD=*****
DB_PASSWORD=*****
API_TOKEN=*****
```

Beneficios:

- Mayor seguridad.
- Configuración flexible.
- Facilita despliegues.
- Reduce riesgos de exposición accidental.

Esta práctica es ampliamente utilizada en arquitecturas basadas en Docker y servicios distribuidos.

Escaneo Automático de Secretos

GitHub ofrece mecanismos de detección automática que permiten identificar credenciales expuestas antes de que lleguen al repositorio principal.

Las capacidades incluyen:

- Detección de claves API.
- Identificación de contraseñas expuestas.
- Alertas automáticas.
- Protección preventiva durante commits.

Estas herramientas forman parte de la estrategia DevSecOps adoptada por el proyecto.

Buenas Prácticas Implementadas

Para fortalecer la protección de secretos se establecen las siguientes recomendaciones:

Gestión Segura de Credenciales

- Utilizar variables de entorno.
- Aplicar rotación periódica.
- Limitar privilegios de acceso.
- Revocar credenciales comprometidas.

Gestión de Repositorios

- Evitar almacenamiento de secretos.
- Revisar commits antes de publicar.
- Utilizar archivos ignorados por Git cuando corresponda.

Gestión Operativa

- Registrar incidentes de exposición.
- Aplicar controles de acceso.
- Realizar auditorías periódicas.

Relación con la Arquitectura OmniSens

La protección de secretos impacta directamente sobre:

- ESP32.
- Broker MQTT.
- Node-RED.
- MariaDB.
- Docker.
- GitHub.
- APIs de integración.

La correcta gestión de credenciales constituye una medida transversal que protege toda la infraestructura tecnológica del proyecto.

Resultado Esperado

Implementar una estrategia de gestión de secretos que permita mantener el repositorio libre de credenciales expuestas y reducir significativamente los riesgos asociados a accesos no autorizados.

Conclusión

La protección de credenciales constituye una práctica esencial dentro de cualquier estrategia moderna de ciberseguridad. La eliminación de secretos hardcodeados, el uso de variables de entorno y la incorporación de herramientas de detección automática permiten fortalecer la seguridad de OmniSens Industrial Suite y reducir la probabilidad de incidentes relacionados con la exposición de información sensible.